

Neste trabalho foi desenvolvido um sistema de chat baseado na arquitetura cliente-servidor utilizando sockets TCP na linguagem Python. O objetivo principal do projeto é demonstrar, de forma prática, como ocorre a comunicação entre aplicações através de uma rede de computadores, utilizando conceitos fundamentais estudados na disciplina de Redes de Computadores.

O sistema permite que múltiplos usuários se conectem a um servidor central e troquem mensagens em tempo real. Para facilitar a visualização e a utilização da aplicação, foi desenvolvida uma interface gráfica simples utilizando a biblioteca Tkinter.

O protocolo TCP (Transmission Control Protocol) é um protocolo orientado à conexão. O modelo cliente-servidor divide a aplicação em duas partes principais:

Servidor

Responsável por:

- Receber conexões dos clientes;
- Gerenciar os usuários conectados;
- Receber mensagens;
- Distribuir mensagens para os demais clientes.

Cliente

Responsável por:

- Conectar-se ao servidor;
- Enviar mensagens;
- Receber mensagens;
- Exibir informações ao usuário através da interface gráfica.

Descrição da Aplicação

A aplicação desenvolvida consiste em um chat de comunicação em tempo real.

Quando o servidor é iniciado, ele fica aguardando conexões em uma porta específica da máquina. Cada cliente que inicia a aplicação realiza uma conexão com o servidor utilizando o endereço IP e a porta configurada.

Após a conexão ser estabelecida, o usuário informa seu nome e pode começar a enviar mensagens. As mensagens recebidas pelo servidor são redistribuídas para todos os demais clientes conectados.

A interface gráfica permite visualizar facilmente as mensagens recebidas e enviadas.

Para o desenvolvimento do projeto foram utilizadas as seguintes tecnologias:

Python, Sockets, Tkinter, TCP/IP, e Threading.

Funcionamento do Sistema

O funcionamento da aplicação ocorre da seguinte maneira:

1. O servidor é iniciado.
2. O servidor abre uma porta para receber conexões.
3. Um cliente executa a aplicação.
4. O cliente estabelece conexão com o servidor.
5. O usuário informa seu nome.
6. O usuário envia uma mensagem.
7. O servidor recebe a mensagem.
8. O servidor encaminha a mensagem para os demais clientes conectados.
9. Os clientes exibem a mensagem recebida na interface gráfica.

Esse processo ocorre continuamente enquanto a conexão permanecer ativa.

Manual do Usuário

Requisitos

Antes de executar o sistema é necessário possuir:

Python 3 instalado;

Sistema operacional Windows, Linux ou macOS.

Executando o Servidor

Abra um terminal na pasta do projeto e execute:

```
python server.py
```

Após a execução será exibida uma mensagem semelhante a:

Servidor iniciado em 0.0.0.0:5000

Isso indica que o servidor está pronto para receber conexões.

Executando o Cliente

Abra outro terminal e execute:

```
python client.py
```

Uma janela do chat será aberta.

Caso deseje simular múltiplos usuários, basta abrir novas instâncias do cliente.

Enviando Mensagens

1. Digite seu nome no campo "Nome".
2. Digite a mensagem desejada.
3. Clique no botão "Enviar".

A mensagem será transmitida para os demais clientes conectados.

Encerrando a Aplicação

Para encerrar a aplicação basta fechar a janela do cliente ou interromper o processo no terminal utilizando:

CTRL + C

O desenvolvimento deste projeto permitiu aplicar diversos conceitos estudados na disciplina de Redes de Computadores, especialmente aqueles relacionados à comunicação cliente-servidor e ao uso de sockets.

A aplicação desenvolvida demonstrou de forma prática como ocorre a troca de informações entre processos distribuídos em rede. Além disso, foi possível compreender melhor o funcionamento do protocolo TCP e a importância dos mecanismos de comunicação confiável.

Resultados Obtidos

Durante os testes realizados foi possível verificar que a comunicação ocorreu corretamente entre múltiplos clientes conectados simultaneamente.

As mensagens enviadas por um usuário foram recebidas pelos demais clientes conectados sem perda de informações. O uso do protocolo TCP garantiu que os dados fossem entregues na ordem correta e de forma confiável.